

Réseaux

Td/Tp n°9

I Rappel des structures de données utilisées

1) Les sockets

A tout socket est associé dans le système un objet ayant la structure *socket*, cette structure n'est pas donnée ici, vous la trouverez décrite dans le fichier standard *<sys/socketvar.h>*. C'est sur un tel objet qu'un descripteur de socket pointera (par une indirection sur une entrée de la table des fichiers ouverts), ces objets sont pour les sockets ce que sont les i-noeuds pour les fichiers. Le type d'un socket détermine la nature de la communication qu'il permet de réaliser, le fichier standard *<sys/socket.h>* contient la définition de constantes symboliques permettant de désigner différents types. On y trouve

SOCK_DGRAM	socket orienté vers la transmission de datagrammes structurés en mode non connecté
SOCK_STREAM	socket orienté vers l'échange de séquences continues de caractères en mode connecté
SOCK_RAW	socket orienté vers l'échange de datagrammes à un niveau bas de protocole (IP par exemple pour pouvoir définir un nouveau protocole situé au dessus).
etc.....	

Un socket doit pouvoir être nommé d'où la notion de domaine (famille), le domaine d'un socket définit le format des adresses utilisées, c'est à l'aide de ces adresses que l'on désigne les sockets. Un certain nombre de domaines sont répertoriés dans le fichier standard *<sys/socket.h>*, on y trouve par exemple

nom symbolique	domaine	format des adresses
AF_UNIX	local	<i>sockaddr_un</i>
AF_INET	Internet	<i>sockaddr_in</i>
AF_NS	Xérox NS	<i>sockaddr_ns</i>
etc.....	le préfixe AF tenant lieu de "Address Family".	

Dans *<sys/socket.h>* on trouve aussi la définition

```
struct sockaddr {
    unsigned short sa_family;    /* address family: AF_XXX value */
    char          sa_data[14];  /* up to 14 bytes of protocol-specific address */
};
```

ceci doit être pris comme une structure générique jamais utilisée.

Pour le domaine des adresses Internet, on trouve dans *<netinet/in.h>* les définitions

```
struct in_addr {
    unsigned long s_addr;        /* 32-bit netid/hostid, network byte ordered */
};

struct sockaddr_in {
    short sin_family;           /* AF_INET */
    unsigned short sin_port;    /* 16- bits port number, network byte ordered */
    struct in_addr sin_addr;    /* 32-bit netid/hostid, network byte ordered */
    char sin_zero[8];          /* unused */
};
```

Pour le domaine UNIX, la famille des adresses est définie dans *<sys/un.h>* par

```
struct sockaddr_un {
    short sun_family;           /* AF_UNIX */
    char sun_path[108];        /* pathname */
};
```

2) Les appels au système

Pour toute la suite, mettre dans les programmes les directives

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

-créer un socket: *socket*

```
int socket( int family, int type, int protocol );
```

La famille (domaine) et le type doivent être choisis dans les ensembles décrits précédemment. Le protocole doit être choisi parmi les protocoles possibles dans le type choisi. En général il n'en n'existe qu'un désigné par l'entier 0 (protocole par défaut). Pour le domaine Internet, on peut utiliser les constantes symboliques **IPPROTO_UDP**, **IPPROTO_TCP** etc... définies dans *<netinet/in.h>*. Le retour est un entier permettant d'atteindre le descripteur ou -1 en cas d'erreur.

-supprimer un socket: *close*

```
int close( int fd );
```

-attacher un socket à une adresse: *bind*

```
int bind( int sockfd, struct sockaddr * myaddr, int addrlen );
```

A la création, le socket n'est connu que du processus l'ayant créé. Un moyen de désigner un socket indépendamment de tout contexte est de lui donner une adresse dont le format est déterminé par son domaine, c'est le rôle de *bind* de réaliser cette association entre le socket passé en premier paramètre et l'adresse qui suit. La structure *sockaddr* est générique, pour un domaine particulier il faudra transtyper le paramètre effectif dont le type est déterminé par le domaine:

```
bind(..., (struct sockaddr *) &par_effectif, sizeof( par_effectif ) );
```

La primitive renvoie l'entier 0 lorsque l'attachement a pu se réaliser sinon -1.

Remarque: un socket étant créé, l'appel à *bind* est facultatif, si le processus créateur émet sur le socket, le système procède alors automatiquement à son attachement à un port quelconque.

-déclarer un service ouvert auprès de son système (mode connecté) : *listen*

```
int listen( int sockfd, int backlog );
```

Un serveur a un rôle passif dans l'établissement d'une connexion; par un appel à *listen* il avise le système auquel il appartient qu'il est prêt à accepter les demandes de connexion des clients. Après réalisation de cet appel, l'entité TCP du système local commence à accepter des demandes de connexions sur le socket passé en paramètre. Le second paramètre indique le nombre maximal de demandes mises en file d'attente. Dans le domaine AF_INET, si le socket spécifié n'a pas été attaché au préalable à une adresse, il y a attachement automatique à l'appel de la fonction. La fonction renvoie la valeur 0 ou -1 selon que l'appel a réussi ou non.

-prendre acte d'une demande de connexion (mode connecté) : *accept*

```
int accept( int sockfd, struct sockaddr * peer, int * addrlen );
```

Le processus serveur prend connaissance d'une nouvelle connexion par un appel à la primitive *accept*. Cette connexion est extraite de la liste des connexions pendantes et un descripteur sur un socket de service dédié à cette nouvelle connexion est renvoyé en retour de la primitive. C'est ce nouveau descripteur qui permettra d'identifier localement la connexion. Le socket dont le descripteur est donné en retour permet donc d'envoyer et de recevoir des caractères sur la connexion. L'adresse du socket du client avec lequel la connexion est établie est décrite en mémoire à l'adresse *peer* si sa valeur est différente de NULL. La longueur de cette adresse est **addrlen*. La fonction retourne -1 lorsqu'une erreur s'est produite, la variable *errno* indique alors la nature de l'erreur.

-demander une connexion (mode connecté) : connect

```
int connect( int sockfd, struct sockaddr * servaddr, int addrlen );
```

Cette primitive permet de demander la connexion du socket dont le descripteur est donné en premier paramètre au socket dont l'adresse est en mémoire à l'adresse *servaddr*. La longueur de cette dernière, en octets étant *addrlen*. Toute connexion entre deux sockets de type SOCK_STREAM est initialisée par cette primitive. Un circuit virtuel est ainsi créé permettant des échanges bidirectionnels. Il n'est pas nécessaire que le socket local (client) se soit préalablement attaché à une adresse, lors de la demande de connexion le système peut réaliser un attachement à un port quelconque. La primitive renvoie 0 lorsque tout se passe correctement sinon -1, la variable *errno* indique alors la nature de l'erreur.

-l'envoi standard de caractères (mode connecté) : write

```
ssize_t write( int sockfd, const void * buff, size_t nbytes );
```

Une fois établie la connexion entre un client et le serveur correspondant, les deux processus peuvent échanger des suites de caractères par l'intermédiaire des deux sockets constituant les deux extrémités de la connexion. A chaque extrémité sont associés deux tampons, un tampon de lecture (réception) dans lequel l'entité TCP écrit les caractères en provenance de l'autre extrémité, un tampon d'écriture (émission) dans lequel l'entité TCP lit les caractères qu'elle doit transmettre à l'autre extrémité. La primitive envoie sur le socket passé en paramètre *nbytes* octets pris à partir de l'adresse *buff*. La primitive retourne le nombre de caractères envoyés ou -1 en cas d'erreur.

-l'envoi spécifique de caractères (mode connecté) : send

```
int send( int sockfd, char * buff, int nbytes, int flags );
```

Les trois premiers paramètres sont identiques à ceux de l'appel correspondant de la primitive *write*. Le dernier paramètre a soit la valeur 0, soit une valeur non nulle, MSG_OOB par exemple dans le domaine AF_INET indiquant que le message contient des données hors bande.

-la réception standard de caractères (mode connecté) : read

```
ssize_t read( int sockfd, void * buff, size_t nbytes );
```

Un tel appel correspond à la demande de lecture sur le socket passé en premier d'au plus *nbytes* caractères qui devront être écrits en mémoire à partir de l'adresse *buff*. La primitive renvoie le nombre de caractères effectivement lus ou -1 en cas d'erreur.

-la réception spécifique de caractères (mode connecté) : recv

```
int recv( int sockfd, char * buff, int nbytes, int flags );
```

Les trois premiers paramètres sont identiques à ceux de la primitive *read*. Le dernier paramètre a soit la valeur 0 ou est une combinaison bit à bit de certaines constantes symboliques comme il a été vu pour la primitive *send*.

-l'envoi d'un message (mode non connecté) : sendto

```
int sendto( int sockfd, char * buff, int nbytes, int flags,  
            struct sockaddr * to, int addrlen );
```

Un processus souhaitant communiquer avec le monde extérieur par l'intermédiaire d'un socket de type SOCK_DGRAM (non connecté), doit réaliser les opérations suivantes. Il faut d'abord réaliser la création du socket, réaliser éventuellement l'attachement de ce socket sur un port convenu ou quelconque, construire l'adresse de son interlocuteur en mémoire, procéder à des émissions ou des réceptions de messages. Chaque demande d'envoi d'un datagramme s'accompagne de la spécification complète de l'adresse de son destinataire et chaque réception d'un message s'accompagne de celle de son émetteur. Il est important de se rappeler que lorsque le domaine utilisé est AF_INET, la communication s'appuie sur le protocole UDP et par suite a exactement les caractéristiques qu'il permet d'assurer. Un appel à la primitive *sendto* correspond à la demande

d'envoi, via le socket identifié localement par le descripteur *sockfd*, du message commençant à l'adresse *buff* et de longueur *nbytes*, à destination du socket dont l'adresse est pointée par *to* et de longueur *addrlen*. Le champ *flags* est à mettre à 0, la primitive renvoie le nombre de caractères effectivement envoyés ou -1 en cas d'erreur.

-la réception d'un message (mode non connecté) : *recvfrom*

```
int recvfrom( int sockfd, char * buff, int nbytes, int flags,  
              struct sockaddr * from, int * addrlen );
```

Un appel à cette primitive permet de lire sur le socket de descripteur *sockfd* un message et d'en récupérer le contenu à l'adresse *buff*. Le paramètre *nbytes* indique le nombre d'octets alloués à cette adresse. Le champ *flags* doit être mis à 0. Un seul message complet est extrait et la lecture ne concerne que ce message qui correspond exactement à un message envoyé par un appel à *sendto*. Dans le cas où le message reçu est de longueur supérieure à l'espace réservé, le message sera tronqué. L'adresse du socket d'émission est récupérée dans l'espace d'adressage du processus récepteur à l'adresse *from* de longueur **addrlen* (dans le cas où l'adresse est différente de NULL). Ne pas oublier d'initialiser ces pointeurs avant l'appel. La primitive est bloquante, elle renvoie le nombre de caractères recopiés ou -1 en cas d'erreur.

3) Quelques outils

utiliser `#include <netinet/in.h>`

-conversion du format local au format réseau

```
unsigned long htonl( unsigned long hostlong );  
unsigned short htons( unsigned short hostshort );
```

-conversion du format réseau au format local

```
unsigned long ntohl( unsigned long netlong );  
unsigned short ntohs( unsigned short netshort );
```

-recopier un nombre donné de caractères

```
bcopy( char * scr, char * dest, int nbytes );
```

-remplir de caractères NULL

```
bzero( char * dest, int nbytes );
```

-comparer un nombre donné de caractères de deux chaînes

```
int bcmp( char * ptr1, char * ptr2, int nbytes );
```

Le retour est 0 lorsque les deux parties sont identiques sinon il est non nul.

-conversion d'une adresse IP donnée sous forme de chaîne (x.y.z.t) en un entier

```
unsigned long inet_addr( char * ptr );
```

-conversion d'une adresse IP donnée sous forme d'entier en une chaîne

```
char * inet_ntoa( struct in_addr inaddr );
```

-obtenir le nom du socket distant (peer) d'une connexion

```
int getpeername( int sockfd, struct sockaddr * peer, int * addrlen );
```

Le socket local admet *sockfd* comme descripteur, en cas d'échec -1 est renvoyé.

-obtenir le nom d'un socket à partir de son descripteur

```
int getsockname( int sockfd, struct sockaddr * addr_ptr, int * addrlen );
```

-obtenir le "hostname"

Déclarée dans <unistd.h>

```
int gethostname( char * name, int namelen );
```

Déclarée dans <netdb.h>

```
struct hostent * gethostbyname( char * hostname );
```

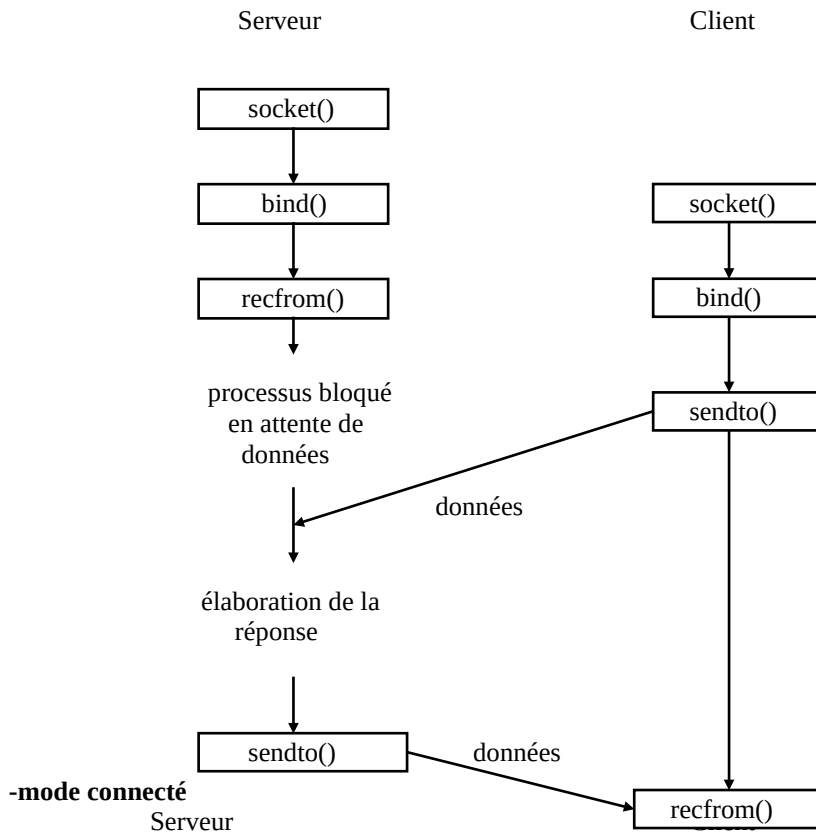
Le retour est un pointeur sur une structure *hostent* définie comme suit:

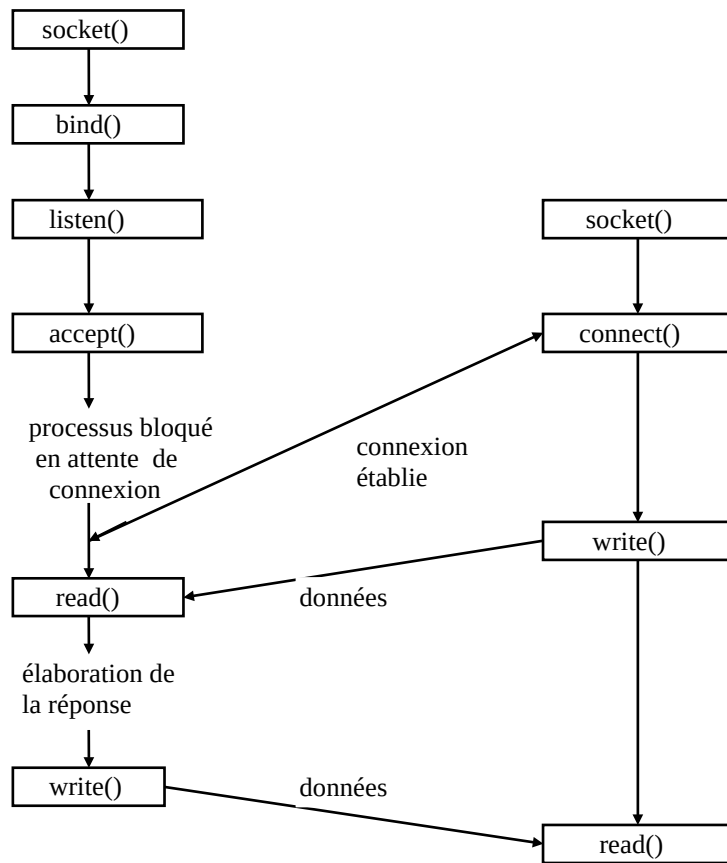
```
struct hostent {  
    char    * h_name;        /* official name of host */  
    char    ** h_aliases;    /* alias list */  
    int      h_addrtype;     /* host address type */  
    int      h_length;       /* length of address */  
    char    ** h_addr_list;  /* list of addresses from name server */  
                                /* a NULL terminates the list */  
};  
#define h_addr h_addr_list[0] /* first address in list */
```

Attention au fait que *h_addr_list[i]* ne sont pas des pointeurs sur caractère mais sur structure de type *in_addr*.

4) Les deux types de fonctionnement du domaine Internet

-mode non connecté:





II Exercice

1) Que fait le code qui suit ?

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <netdb.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>

#define SERVER_PORT 4500
#define MAX_CONNECTION_NB 16
#define MAX_LEN_MSG 33

main() {
    int sockfd;
    int err;
    char msg[MAX_LEN_MSG];

    sockfd= create_socket();
    while ( 1 ) {
        int pid;
        struct sockaddr_in conn_addr;
        int length= sizeof( struct sockaddr_in );
        int des= accept( sockfd, ( struct sockaddr * ) &conn_addr, &length );
        if ( des < 0 ) {
            perror( "Erreur dans l'appel de accept :" );
            exit( 1 );
        } /* if */
        printf( "Nouvelle connexion établie %d \n", des );
        switch( pid = fork() ) {
            case -1 : perror( "Erreur dans l'appel de fork :" );
                    break;
            case 0 : err= read( des, msg, MAX_LEN_MSG - 1 );
                    if ( err < 0 ) {
                        perror( "Erreur dans read :" );
                        exit( 1 );
                    } /* if */
                    printf( "Le message reçu est : %s\n", msg );
                    strcpy( msg, "Message reçu, client!" );
                    err= write( des, msg, MAX_LEN_MSG - 1 );
                    if ( err < 0 ) {
                        perror( "Erreur dans write :" );
                        exit( 1 );
                    } /* if */
                    exit ( 0 );
                    close( des );
                    break;
            default :close( des );
                    break;
        } /* switch */
    } /* while */
} /* main */

int create_socket() {
    int sockfd;
    struct sockaddr_in sock_name;
    int err;
```

```

sockfd= socket( AF_INET, SOCK_STREAM, 0 );
if ( sockfd == -1 )
    {
        perror( "Erreur dans la création d'un socket :");
        exit( 1 );
    }
    /* if */
bzero( ( char * ) & sock_name, sizeof( sock_name ) );
sock_name.sin_port= SERVER_PORT;
err= bind( sockfd, ( struct sockaddr * ) & sock_name, sizeof( sock_name ) );
if ( err == -1 )
    {
        perror( "Erreur lors de l'appel à bind :");
        exit( 1 );
    }
    /* if */
err= listen( sockfd, MAX_CONNECTIO_NB );
if ( err == -1 )
    {
        perror( "Erreur lors de l'appel à listen :");
        exit( 1 );
    }
    /* if */
return sockfd;
}
/* create_socket */

```


2) Que fait le code qui suit ?

```
/* reprendre les mêmes directives d'inclusion que pour la question 1) */
#define SERVER_PORT 4500
#define MAX_LEN_MSG 33
main( int argc, char ** argv )    {
    int sockfd;
    int i, err;
    struct sockaddr_in sock_name;
    int length= sizeof( struct sockaddr_in );
    char * hostname;
    struct hostent * server;
    char msg[ MAX_LEN_MSG ];

    if ( argc != 3 )    {
        fprintf( stderr, "Erreur dans le nombre d'arguments de la ligne de commande \n");
        exit( 1 );
    }    /* if */
    hostname= argv[ 1 ];
    if ( strlen( argv[ 2 ] ) > MAX_LEN_MSG - 1 )    {
        fprintf( stderr, "L'argument de la chaîne de commande est trop long \n");
        exit( 1 );
    }    /* if */
    strcpy( msg, argv[ 2 ], MAX_LEN_MSG - 1 );
    server= gethostbyname( hostname );
    if ( ! server )    {
        fprintf( stderr, "Erreur dans la détermination du hostname \n");
        exit( 1 );
    }    /* if */
    printf( "server hostname = %s\n", server->h_name );
    printf( "Internet address = %s\n", inet_ntoa( *server->h_addr );
    bzero( ( char * ) & sock_name, sizeof( sock_name ) );
    sock_name.sin_family= AF_INET;
    bcopy( ( char * ) server->h_addr_list[0], ( char * ) & sock_name.sin_addr.s_addr,
                                                server->h_length );

    sock_name.sin_port= SERVER_PORT;
    sockfd= socket( AF_INET, SOCK_STREAM, 0 )
    if ( sockfd == -1 )    {
        perror( "Erreur lors de la création d'un socket :" );
        exit( 1 );
    }    /* if */
    err= connect( sockfd, (struct sockaddr *) & sock_name, sizeof( sock_name ) );
    if ( err == -1 )    {
        perror( "Erreur lors de la demande de connexion :" );
        exit( 1 );
    }    /* if */
    err= write( sockfd, msg, MAX_LEN_MSG - 1 );
    if ( err < 0 )    {
        perror( "Erreur lors de wтите :" );
    }
}
```



```

exit( 1 );

}          /* if */
msg[ 0 ]= 0;
err= read( sockfd, msg, MAX_LEN_MSG - 1 );
if ( err <= 0 ) {
    perror( "Erreur lors de read : " );
    exit( 1 );
}          /* if */
printf( "voila le message reçu : %s\n", msg );
close( sockfd );
printf( "Terminaison du client\n" );          exit( 0 );
}          /* main */

```

MISE EN OEUVRE

L'objectif de ce T.P est de réaliser des échanges "simples" de type Client/Serveur, entre processus distants.

On suppose une organisation qui dispose d'un réseau téléphonique interne organisé autour d'un autocommutateur. Un fichier texte fictif représente l'annuaire des abonnés à ce réseau. Il contient autant de lignes que d'employés de l'organisation, dans chaque ligne apparaît un nom d'employé et son numéro de poste téléphonique séparés par un espace. Le numéro de poste est constitué de 4 chiffres.

On souhaite mettre en place, sur un site UNIX, un serveur qui permet, à partir d'une requête composée d'un nom d'employé émise par des processus clients s'exécutant sur des sites distants, de retourner au client le numéro de poste correspondant à cet employé.

La démarche suivante permet d'aboutir progressivement à la solution souhaitée tout en étudiant diverses alternatives :

1°/ Un client et un serveur en mode datagramme.

Ecrire les programmes correspondant à un processus client et à un processus serveur utilisant le mécanisme des sockets en mode datagramme. Le processus client sera lancé avec en argument le nom de la machine distante sur laquelle réside le processus serveur.

2°/ Un client et un serveur en mode connecté.

Même travail, mais cette fois, le mécanisme des sockets en mode connecté est utilisé.

3°/ Serveur concurrent en mode connecté.

Transformer le code du serveur de l'étape précédente de façon à ce qu'il effectue une création d'un processus fils pour lui faire effectuer le traitement associé à une connexion.

4°/ Serveur multi-ports en mode datagramme.

A nouveau en mode datagramme, écrire un processus serveur « multi-ports » qui effectue une surveillance active (polling) sur chacun des ports qu'il gère et traite une requête dès qu'elle est détectée sur un de ces ports. On pourra tester au moins le cas de deux clients. On utilisera la primitive select.

REMARQUE : n'hésitez pas à consulter le man.. tout n'est pas dans la synthèse précédente...

Extrait du code de traitement sur le fichier texte pour la recherche de la correspondance entre un nom reçu et l'éventuel numéro de poste correspondant :

```
#define MSG_MAX_SIZE 20
#define LG_MAX_SIZE 40

charNomRecu[MSG_MAX_SIZE];
char Ligne [LG_MAX_SIZE] ;
char Tel [5] ;
FILE *fp;
Unsigned short Trouve = 0 ;

fp = fopen("fictel","r") ;
/* Recherche du numéro de poste associe au nom */
while (fgets (Ligne, LG_MAX_SIZE, fp) && ! Trouve)
{
    Ligne [strlen(Ligne)-1] = '\0'; /* Suppression \012 */
    if (!strcmp(NomRecu, Ligne+5)) /* Trouve */
    { Trouve = 1;
      strncpy(Tel,Ligne,4);
    }
}

fclose(fp);
if (!Trouve) strcpy(Tel,"????");
```

Exemple de fichier texte annuaire

```
6768 messaoudi
6929 kaidi
6024 abdellaoui
6123 elomari
6024 chiba
6127mahfoud
6769 sekkaki
6024 moussaid
```